



emit으로 데이터 올려보내기

slot으로 콘텐츠 내려보내기

부모→자식(props), 자식→부모(emit) 흐름한 번에 이해하기



학습 목표

Props로 데이터 내려보내기 복습

Emit으로 이벤트 올려보내기 실습

코드 예제 그대로 따라하며 구현하기



이번 실습 파일 구조



App.vue

Parent



components/



MyCard.vue

Child

Props 복습: 부모 → 자식 데이터 전달



핵심 개념

부모 컴포넌트가 자식 컴포넌트에게 데이터를 전달할 때 사용하는 단방향 데이터 흐름 방식입니다.



자식 (Child)

defineProps로 받을 데이터의 타입과 기본값을 선언합니다.



부모 (Parent)

자식 태그에 HTML 속성처럼 값을 전달합니다.

* 동적 데이터는 : (v-bind) 사용

components/MyCard.vue (자식)

```
// 1. props 선언 (JS 영역)
const props = defineProps({
  title: { type: String, default: '기본 제목' },
  count: { type: Number, default: 0 },
  isActive: { type: Boolean, default: false },
})
```

App.vue (부모)

```
<!-- 2. 자식에게 props 전달 (Template 영역) -->
<MyCard
  title="Vue 강의"
  :count="3"
  is-active <!-- Boolean true 전달 -->
/>

<MyCard
  :count="0"
  :is-active="false"
/>
```

Emit 개념: 자식 → 부모 이벤트 전달



핵심 개념

자식 컴포넌트에서 부모 컴포넌트에게 특정 사건(Event)이 발생했음을 알릴 때 사용하는 메커니즘입니다.

↓ Props

부모가 자식에게 데이터를 내려줌

↑ Emit

자식이 부모에게 신호/메시지를 올림



동작 방식

자식은 emit() 함수로 이벤트를 '발사'하고,
부모는 v-on(@) 디렉티브로 이를 '수신'합니다.

Convention Tip:



이벤트 이름은 HTML 속성처럼 kebab-case 사용을 권장합니다. (예: send-message)

자식 컴포넌트 (Script)

```
// 1. 보낼 이벤트 목록 선언
const emit=defineEmit([ 'send-message' ])

// 2. 이벤트 발사 (함수 안에서 호출)
const notifyParent= () => {
  // 첫 번째 인자: 이벤트 이름, 두 번째: 데이터(Payload)
  emit('send-message', '안녕하세요!')
}
```

부모 컴포넌트 (Template)

```
←!— 3. 이벤트 수신 및 핸들러 연결 —→
<MyCard
  @send-message="handleSendMessage"
/>

←!— 자식이 emit하면 handleSendMessage 함수가 실행됨 —→
```

자식 컴포넌트 구현 (Props + Emit)



Template (UI 구조)

데이터를 화면에 표시하고, 사용자 동작(버튼 클릭)을 감지합니다. @click="notifyParent"로 클릭 이벤트를 연결합니다.



Script Setup

1. Props: 부모가 준 데이터를 받습니다. 2. Emits: 부모에게 보낼 이벤트를 정의합니다. defineEmits(['send-message'])



이벤트 발송 로직

버튼이 클릭되면 emit() 함수를 실행하여 부모에게 메시지 (payload)를 전달합니다.

MyCard.vue - Template

```
<template>
  <div class="card-box">
    <h2>{{ title }}</h2>
    <p>좋아요 수: {{ count }}</p>

    <!-- 버튼 클릭 시 함수 실행 -->
    <button type="button" @click="notifyParent">
      부모에게 메시지 보내기
    </button>
  </div>
</template>
```

MyCard.vue - Script Setup

```
<script setup>
  // 1. Props 선언 (기존 유지)
  const props = defineProps({ title: String, count: Number })

  // 2. Emit 선언 (이벤트 이름 등록)
  const emit = defineEmits(['send-message'])

  // 3. 버튼 핸들러: 부모에게 데이터 전송
  const notifyParent = () => {
    const msg = `자식: "${props.title}", 좋아요 ${props.count}개`
    emit('send-message', msg) // 이벤트명, 데이터
  }
</script>
```

부모 컴포넌트 구현: App.vue



구현 포인트

자식(MyCard)이 보낸 이벤트를 수신하고, 전달받은 데이터를 화면에 업데이트합니다.



이벤트 수신 (v-on)

@send-message로 자식 이벤트를 감지하고 핸들러 함수를 연결합니다.



핸들러 함수 정의

매개변수 payload를 통해 자식이 보낸 데이터를 수신합니다.



상태 업데이트

반응형 변수(ref) lastMessage 값을 갱신하여 UI를 다시 렌더링합니다.

```
App.vue

// 1. 이벤트 핸들러: payload로 데이터 수신
const handleMessage = (payload) => {
  console.log('부모가 받은 메시지:', payload)
  lastMessage.value = payload
}

</script>

<template>
  <main style="text-align:center; margin-top:1.5rem;">
    <h1>Props + Emit 데모</h1>

    <!-- 2. 자식 컴포넌트에 이벤트 리스너 연결 -->
    <MyCard
      title="Vue 강의"
      :count="3"
      is-active
      @send-message="handleSendMessage"
    />
  </template>
```

데이터 흐름 다이어그램



양방향 소통 과정



데이터 전달 (Down)

부모(App)가 자식에게 props를 통해 데이터를 내려보냅니다.

예: title, count, isActive



이벤트 발신 (Up)

자식(MyCard)이 emit 함수로 이벤트를 발생시켜 부모에게 알립니다.

예: emit('send-message', '내용')



상태 갱신 (Update)

부모는 이벤트를 감지하여 핸들러 함수를 실행하고, 자신의 상태(ref)를 업데이트합니다.



구현 시 확인 포인트

이벤트 이름 일치 여부 (emit('name') vs @name)

Payload 데이터 형식 (문자열, 객체 등) 확인

부모 핸들러 함수의 인자로 데이터가 잘 들어오는지 확인

Mission: 부모에게 알림 보내기



1/2 진행 단계



이벤트 선언하기 (자식)

MyCard.vue에서 `defineEmits(['send-message'])`를 추가하여 사용할 이벤트를 정의하세요.



버튼 만들고 이벤트 발사 (자식)

템플릿에 버튼을 추가하고 클릭 시 `emit('send-message', '내용')`을 호출하는 함수를 연결하세요.



이벤트 듣기 (부모)

App.vue에서 컴포넌트 태그에 `@send-message="handler"` 속성을 추가하여 연결하세요.



확인하기

버튼을 클릭했을 때 부모 컴포넌트의 화면이나 콘솔에 메시지가 잘 출력되는지 확인하세요.

LEVEL UP!



도전 과제 (선택)

객체 전달하기: 단순 문자열 대신 `{ title: '...', count: 5 }` 형태의 객체를 payload로 보내보세요.

다중 이벤트: reset 이라는 이벤트를 하나 더 만들어서 카운트를 0으로 초기화 해보세요.



디버깅 Tip

Vue DevTools를 사용하면 컴포넌트에서 발생한 이벤트(Emit)와 전달된 Payload를 타임라인 형태로 시각적으로 확인할 수 있습니다.

Slot 개념: 엘리먼트 & 템플릿 전달



통신 방식 비교

데이터 전달 방향과 목적에 따라 적절한 방식을 선택해야 합니다. Slot은 단순 데이터가 아닌 UI 구조 자체를 전달합니다.

Props



부모 → 자식 : 데이터(값) 전달

Emit



자식 → 부모 : 이벤트(신호) 전달

Slot



부모 → 자식 : 엘리먼트(템플릿) 전달

App.vue (부모에서 사용)

←!— 태그 사이에 내용을 넣음 →

```
<AlertBox>
```

나쁜 일이 일어났습니다.

```
</AlertBox>
```

AlertBox.vue (자식 컴포넌트)

```
<template>
```

```
<div class="alert-box">
```

```
<strong>이것은 데모용 에러입니다.</strong>
```

←!— 부모가 넣은 내용이 이곳에 표시됨 →

```
<slot /><!-- 자리표시자
```

```
</div>
```

```
</template>
```

i Tip: 재사용 가능한 UI 컴포넌트(모달, 카드, 레이아웃 등)를 만들 때 필수적입니다.

AlertBox & HelloWorld 코드 예제



1. AlertBox.vue (기본 Slot)

```
components/AlertBox.vue

<template>
  <div class="alert-box" >
    <strong> 데모용 에러 </strong>

    <!-- 부모 컨텐츠가 올 자리 -->
    <slot />
  </div>
</template>

<script setup>
  // 로직 없음 (단순 래퍼)
</script>

<style scoped>
.alert-box {
  border :1px solid #f5a5a5 ;
  background :#fff5f5 ;
  padding :10px ;
}
</style>
```

2. HelloWorld.vue (Props+Slot)

```
components/HelloWorld.vue

<script setup>
  const props = defineProps ({
    msg : { type :String },
    number : { type :Number }
  })
</script>

<template>
  <div class="hello" >
    <!-- Props 출력 -->
    <h1> {{ props.msg }} </h1>
    <h2> {{ props.number }} </h2>

    <!-- Slot 위치 -->
    <slot></slot>
    <p> KOPO </p>
  </div>
</template>
```

3. App.vue (부모 컴포넌트)

```
App.vue

<template>
  <main>
    <h1> Slot 데모 </h1>

    <!-- 1) AlertBox 사용 -->
    <AlertBox>
      나쁜 일이 일어났습니다.
    </AlertBox>

    <!-- 2) HelloWorld 사용 -->
    <HelloWorld
      msg="하하하하"
      :number="123"
    >

    <!-- 이 내용이 slot으로 -->
    <h1> 엘리먼트 </h1>
  </HelloWorld>
</main>
</template>

<script setup>
  import AlertBox from './AlertBox.vue'
  import HelloWorld from './HelloWorld.vue'
</script>
```

Slot 실습 미션: 부모 내용을 자식에게



≡ 진행 단계

AlertBox 컴포넌트 생성

components/AlertBox.vue 파일을 생성하고, 템플릿 내부에 `<slot />` 태그를 추가하세요.

App에서 AlertBox 사용

App.vue에서 AlertBox를 import하고, 태그 사이에 원하는 문구를 넣어보세요. 예:
`<AlertBox>오류가 발생했습니다!</AlertBox>`

HelloWorld에 Slot 추가

기존 HelloWorld.vue 파일의 템플릿 중간에 `<slot></slot>`을 추가하세요.

✓ 확인 포인트

부모가 적은 내용이 자식의 `<slot>` 위치에 정확히 나타나는가?

Props(msg, count)와 Slot 콘텐츠가 함께 렌더링될 때 순서가 맞는가?

브라우저 개발자 도구에서 DOM 구조가 어떻게 생성되었는지 확인했는가?

💡 실습 팁

Slot 안에는 단순 텍스트뿐만 아니라, 복잡한 HTML 구조나 다른 컴포넌트도 넣을 수 있습니다.

자식 컴포넌트에 `<slot>`이 없으면, 부모가 내용을 전달해도 화면에 보이지 않습니다.